Low-cost anonymous reputation update for IoT applications

Alex Shafarenko
University of Hertfordshire, Hatfield AL10 9AB, UK

Abstract

This paper presents a novel zero-trust anonymous reputation update protocol for IoT crowdsensing applications. We use a suite of cryptographic functions to achieve anonymity, including unlinkability of sensing reports to the principals that submit them, and of reports to one another, while enabling the infrastructure to reliably quantify the degree of trust expressed as a reputation level. The protocol is low-cost for the anonymous participant due to the use of cheap standard functions: low-exponent modular exponentiation and cryptographic hashing, which makes it quite suitable for IoT. Despite that, uniquely it does not require anonymous clients to trust the servers even if the latter could collude in an attempt to de-anonymise the former. Furthermore under the protocol, selective onboarding does not require trust either, as the gate keeper issues tickets assuredly blindly after examining the real user's credentials. On the other hand, reputation values versus time can be, and are, linked, enabling statistical analysis for accurately assessing reports without jeopardising anonymity. The protocol has been benchmarked on an off-the-shelf microcontroller broadly used in IoT solutions, with the latency measured at less than 30ms per round for the 4K RSA modulus that NIST recommends until the year 2035. The power drain has been calculated to be low enough for an implementation as a key fob using actual components and their manufacturer guaranteed data, with an embedded disc battery supporting the solution for in excess of 3 years.

1 Introduction

Crowdsensing is an important technique of gathering information in smart cities especially when automotive platforms are used to move sensors around(Ji et al., 2023). An unfortunate side effect of their use is the disclosure of the vehicle position in time as the sensing methods are usually short-range, so whenever a sensing report is submitted it has to include the coordinates of both the sensing target and the vehicle: those would be nearly identical. Since crowdsensing is done by willing participants, it is important to reassure them that their movement across the urban environment is kept confidential to ensure the privacy of their journeys.

Privacy is indeed protected by law, but so is property, and yet we require effective locks on our doors. A user would be more willing to join a crowdsensing arrangement if they are presented (perhaps indirectly, via an expert community) with a proof that the user identity cannot be correlated with their position at various times. If the number of participating vehicles is large, the most the data controller is allowed to know is that a sensing report has been received from a vehicle in a certain location at a certain time, but not which vehicle or who was in it or even what other sensing reports the vehicle has sent in the past (a property which we will call report *unlinkability*).

In other words, the user requires anonymity: not by promise, but by design. The difficulty is in that not all reports are intrinsically reliable: some may belong to malicious users trying to send a false report for their own benefit or even for the benefit of a third party. Other vehicles may have faulty sensors, and the faults could be either steady or irregular. Anonymity and unlinkability, while helping the user to avoid privacy violations, could make it more difficult for the system to identify a malicious agent. Therein lies the principle tension of any anonymous reputation update scheme.

Since the publication of (Wang et al., 2013) an effective operational structure of the solution has been in place: a registration authority server, which stores each client's reputation history under the client's pseudonym and which can issue the client with its reputation certificate; and a reputation server that receives that certificate along with a fresh sensing report from the client and which issues a reputation-update coupon that the client returns to the authority for reputation update. The challenge that still remains is to ensure that neither the coupon nor the certificate can be linked with the pseudonym: this would prevent the servers' possible collusion in linking the client's reports together. Another challenge is to ensure that the client's part of the protocol is low-cost as we are interested in IoT applications, such

as smart sensors. There are other important threats and constraints we will discuss later on, and the paper provides a complete solution to all these issues.

Specifically, our original contributions to knowledge are as follows:

- a new anonymous reputation update protocol with a small resource footprint, suitable for members of a mobile, low-power IoT swarm;
- a zero-trust solution for anonymity protection which cannot be defeated by the collusion of the servers;
- the protocol supports a zero-trust implementation of the servers since no secrets other than the server's private key for signing and no random data are used in computation of protocol steps on the server side; thus the client is reassured that the server can not secretly store and piggy-back identity information on protocol messages even if a server can obtain it on a side channel.
- for conveying reputation levels in coupons and certificates we have proposed a new variant of the RSA blind signature scheme with public metadata; the new scheme is specialised for the case when the metadata is chosen by the signer alone and from a small set, which does not necessitate full-domain exponentiation;
- based on the ideas of Guy-Fawkes protocols (Anderson et al., 1998) and Winternitz chain, we developed a pseudonym authentication method that does not require an encrypted channel and, by extension, Diffie-Hellman type of channel protection, which would typically cost as much as full-domain exponentiation;
- we have shown that the client part of our protocol can be supported by an autonomous, sealed, battery-powered device the size of a key fob, and that the power budget of such a device would last for at least 3 years; the protocol latency for the client is shown to be less than 30ms, with the benchmarking and feasibility study done using broadly available off-the-shelf components.

In the sequel we present our solution. The next section describes the setting in which the Anonymous Reputation Update Protocol (ARUP) operates and introduces the threat model that we assume in our design. Section 3 introduces three contributing techniques: the Winternitz chain, which makes it possible to have repeated authentication of the anonymous user without needing protected channels or expensive cryptography to make public channels confidential; a Guy Fawkes technique of message authentication; and a low-cost version of multiexponent RSA signature for blind signing with public metadata, without agreeing the metadata first. Section 4 defines ARUP and presents its 8 steps in full detail with explanations and security analysis, including post-quantum considerations. Section 5 briefly introduces two important optimisations. Section 6 presents benchmarking of the protocol on a real IoT platform (ESP32) and some protocol power consumption calculations in an experimental key-fob prototype. Section 7 surveys related work, and finally there are some conclusions.

2 Problem setting

Our architecture, described below, bears some similarity to the proposal in (Wang et al., 2013), which remains relevant today as it inspires further research (see, for example, (Sadiah and Nakanishi, 2022), (Sadiah and Nakanishi, 2024)) and has a clear separation of concerns between reputation update and report assessment.

Assuming a swarm of automotive sensors (without loss of generality) we have four principals in the setting, two of which are replicated (the Reputation Server and the swarm node, or client).

The Anonymous Reputation Update is performed by four principals:

Gate Keeper (GK) The purpose of the GK is to control who may or may not join the swarm from outside. A human candidate may present to the GK their real-world identity, e.g. for paying a fee, without risking the future anonymity. The result of a transaction with the GK is an *admission ticket*, which is anonymous and is signed blindly by the GK, i.e. it would be impossible to link the ticket with the human on behalf of whom it was issued. The ticket guarantees admission by the RA, which we describe next.

Registration Authority (RA) has a dual purpose. First, it is used to hold every client's current reputation without access to the client's real identity or any situation reports the client may submit

from time to time for evaluation as it moves about. The current reputation is provided to the client in the form of a one-time, anonymous certificate. Secondly, the RA is there to update the client's reputation based on coupons that the client obtains from the RS, which we describe next.

Reputation Server (RS) Has a single function. It receives a certificate and a situation report from the client for evaluation. The RS evaluates the report alongside other reports it may have received and forms a view of the situation taking into account the reputation levels of the clients that sent those reports. Then it evaluates the client's report for its correspondence with that view and, based on that, it determines the level of trust for the report and proposes the client's new reputation. The reputations (old and new) then form the public part of the coupon that the RS blindly signs and returns to the client. There can be more than one RS, but generally only one RA.

A swarm node is a moving platform which has the ability to sense the environment to form a situation report. The swarm node (which we call client for short) interacts with the RA to obtain its certificate, then sends it along with the situation report to an RS. The RS responds with a one-time, anonymous coupon (even if the client's reputation has not changed which is the most common situation) and the client returns the coupon to the RA, which updates the client's reputation and which issues a new certificate for the client to continue on to the next round of the protocol.

Security objectives The protocol guarantees that the Registration Authority would not be able to determine *where* and *when* the client has visited even though the situation report is referenced to the location and time of measurement (this is key for establishing its relevance and trustworthiness), even if the RA and RS were to collude. It follows that the coupon must contain no data hidden from the client, in particular the reputation update data must be standardised and in the plain.

Further objectives include

1. to prevent the client from "forgetting" an unfavourable reputation update by destroying the coupon. We are interested in a protocol with an invariant:

$$\#\text{ of tickets} = \#\text{ of certificates} + \#\text{ of coupons},$$

   where only spent tickets, certificates that have not been blacklisted and unredeemed coupons are counted;

2. to prevent certificate trading whereby colluding clients can obtain a reputation increase by using a certificate not issued to themselves;

3. to minimise the computational cost of the protocol for the client (but not necessarily the servers) to enable low-power devices to participate in the process of anonymous reputation update directly, without a trusted intermediary;

4. to achieve the previous objectives without making extraordinary demands on the communication infrastructure.

2.1 Threat model

1. The RA and RS may collude to attempt correlating the client's time-referenced situation reports (which include its position) and thus de-anonymise the client.

2. All communication between the client and a server is public; furthermore, any of the messages may be intercepted, including man-in-the-middle attack. In a broadcast environment this could be in the form of jam-spoof attack, whereby a message is intercepted and jammed at the same time, so no listener can receive it except the attacker, and then new messages may be broadcast by it, based on the intercepted data. All communication channels possess *eventual reliability*, i.e. they deliver messages intact after a finite number of retransmissions.

3. Clients may go rogue and seek to undermine high-reputation colleagues, either individually or in groups. In particular they may attempt to trade reports, reputation certificates or reputation update coupons. More on this in the sequel.

If the same valid message is sent to a server again, our protocol requires it to be ignored with the original server response repeated, even if the original received message has changed the server's state. Coupled with the channel's eventual reliability this guarantees that the client resending a valid message will eventually receive the server's genuine response.

3 Basic techniques

In this section we will discuss three contributing techniques that secure anonymous reputation update against the threats indicated above.

3.1 Winternitz chain

A chain of credentials $\{z_i\}_{i\in[0,L]}$ is a secret sequence of bit strings for which the following two properties hold. For any $i < L$,

1. If z_i for some i is known to a principal outside the client, the principal **can** validate a claimed z_{i+1} quickly and at a small cost.
2. If z_i for some i is known to a principal outside the client, the principal **cannot** find a value of z_{i+1} that passes validation given realistic resources and the time shorter than the maximum interval between the client's visits to the RA.

A chain of credentials is useful as a means to support at low cost prover-verifier authentication without sharing and maintaining a secret. The prover simply reveals z_{k+1} to the verifier that knows z_k. The value of z_{k+1} is validated and kept in store for the next authentication session. All revealed z_i are immediately published, so no secret need be kept. Unfortunately, the session requires the communication to be confidential, otherwise an adversary can insert itself between the prover and verifier as a man-in-the-middle. The attack is probabilistically successful since the order of message arrivals is not guaranteed. A confidential channel without a persistent shared secret is, although compatible with anonymity, is as costly as Diffie-Hellman protocols, requiring full domain exponentiation. On the other hand chain based authentication has its storage demands. We will address the necessary countermeasures later.

The chain of credentials can be implemented using a cryptographic hash function $\bar{H} : B^* \to B^m$ that returns an m-bit string when it is applied to an arbitrary bit string. Assume that the client selects x_L at random and the rest of the chain is defined by the following recurrence relation for $0 \leq i < L$:

$$z_i = \bar{H}(z_{i+1})\,.$$

The above construction is commonly referred to as a Winternitz chain in the literature on hash-based signature systems, but no authoritative source is known where it is defined. The security properties of the Winternitz chain follow from the properties of the hash function used to create it. If the function is pre-image resistant, property 2 above is assured, while property 1 is to do with an efficient hash implementation, which is generally available to IoT devices at low cost.

3.1.1 Message piggy-backing

As mentioned earlier, even though a chain of credentials is a useful tool for authenticating a client without sharing a secret with a server, in the presence of an adversary session-based protocols require cryptographic scaffolding that challenges the computational budget of a low-power IoT device. The purpose of a session is to create a secure authenticated environment for the parties to exchange data in. Fortunately, it can also be achieved in a two-phase request-response sessionless protocol, which does not require costly cryptography.

Phase 1

client: send **Ph1**($M = M_i$, $z = z_i$, $\alpha = \bar{H}(z_{i+1}||M_i)$);

server: receive **Ph1**(M, z, α). Send **Ph2**(receipt = sig$(M||z||\alpha)$). Check if a queue is bound with z; if no queue bound, bind an empty queue. Put (M, α) on the queue bound with z;

Phase 2

client: receive **Ph2**(receipt). Check receipt = sig$(M_i||z_i||\bar{H}(z_{i+1}||M_i))$.
If true, send **Ph2**$(z = z_{i+1})$, otherwise back to Phase 1;

server: receive **Ph2**(z).
Compute $z^* = \bar{H}(z)$. Check a queue is bound with z^*. If no queue, NACK. Otherwise scan queue from start for first item matching $(M, \bar{H}(z||M))$.
If not found, NACK. Otherwise, M is authentic; send ACK, delete the queue, and process M.

Here as before all communication is public. The key idea of the protocol is that in Phase 1 the client sends to the server authenticator α, which is a hash[1] of the message (sent along in the plain, together with z_i) and the currently secret *next* chain point z_{i+1}. The knowledge of α does not help an adversary learn z_{i+1} ahead of time, since the hash function is one-way. The server acknowledges the receipt of M, z and α using a standard digital signature with cheap verification (we chose RSA with a low public exponent). To thwart the man-in-the-middle attack, which we will consider next, the tuple (M_i, α) is put on a queue bound with z_i. If it is already on the queue there is no need to enqueue a replica, but it would do no harm either.

At the beginning of Phase 2, the client receives the receipt and satisfies itself that it is genuine. It then publicly reveals z_{i+1}. The server hashes that value to obtain the previous chain point, z_i and examines the queue associated with it. Multiple outcomes are possible outcomes:

$\bar{H}(z)$ **is not a key in KVS.** Either z was received in error or it is a DOS attack, ignore either way.

$\bar{H}(z)$ **is a key in KVS but there is no queue bound to it.** z is a retransmission, ignore.

there is a queue bound with $\bar{H}(z)$. This is normal for Phase 2. If the queue has more than one item on it, either there was a transmission error, or a man-in-the-middle attack is in progress; continue.

In the last case do the following. Starting from the least recent queue element work towards the head of the queue, checking every queue item (M', α') to see whether

$$\alpha' = \bar{H}(z||M')\,.$$

The first time the above check succeeds, the server treats the corresponding string M' as the genuine message M_i from the owner of the Winternitz chain $\{z_i\}$. Notice that the adversary is unable to prevent the server's reception of the genuine Phase 1 transmission since the client will keep retransmitting it until it receives the server's receipt, which is signed and so cannot be forged by the adversary.

3.1.2 Security analysis

The above two-phase protocol defeats the man-in-the-middle attack of the following kind:

- The adversary intercepts z_{i+1} and prevents it from being received by the server at the same time by jamming the channel.
- It then forms its own Phase 1 transmission on behalf of the client: $\mathbf{Ph1}(M^*, \bar{H}(z_{i+1}), \bar{H}(z_{i+1}||M^*))$ and follows that by $\mathbf{Ph2}(z_{i+1})$, expecting the server to accept M^* as a genuine message from the chain owner.

The attack fails because the attacker needs to wait until the client sends its genuine Phase 2 transmission. Whether or not it is jammed for the server makes no difference since the client only sends it when it has received the server's valid receipt, which means that the genuine (M, α) has already been enqueued by the time the adversary obtains the Phase 2 message. The attacker's Phase 1 message could only be second on the queue, with a valid genuine item ahead of it. So the attacker's message M^* will not be accepted.

If the attacker could transform a valid Phase 1 message to a different *valid* transmission $\mathbf{Ph1}(M^*, z_i, \alpha^*)$ for the same z_i, this would work, since although the receipt to it would not satisfy the genuine client, the attacker's Phase 1 transmission will result in the server's binding z_i with a queue on which the item (M^*, α^*) would be first. The unsatisfied client would, of course retransmit, but even if no further jamming is encountered the genuine item (M, α) would not be first on the queue. However, to find an α^* such that $\alpha^* = \bar{H}(z_{i+1}||M^*)$ is practically impossible without finding z_{i+1} from M and $\alpha(z_{i+1}||M)$. The latter is also practically impossible due to the one-wayness of the cryptographic hash function.

3.1.3 Guy-Fawkes protocol: general piggybacking

In our analysis so far we did not use the fact that z_{i+1} and z_i belong to a chain. In fact this method can be used for a single transaction, using a random nonce ϕ playing the role of z_{i+1}, with z_i replaced by $\bar{H}(\phi)$. Here is the updated protocol:

Phase 1

[1] All hash arguments here and below are fixed public length, so length attacks on any hash function, whether short or full-domain, are inhibited.

client: Choose random ϕ. Send **Ph1**$(M, \nu = \bar{H}(\phi), \alpha = \bar{H}(\phi||M))$;

server: receive **Ph1**(M, ν, α). Send **Ph2**(receipt = sig$(M||\nu||\alpha)$). Check if a queue is bound with ν; if no queue bound, bind an empty queue. Put (M, α) on the queue bound with ν;

Phase 2

client: receive **Ph2**(receipt). Check receipt = sig$(M||\bar{H}(\phi)||\bar{H}(\phi||M))$.
If true, send **Ph2**$(\Phi = \phi)$, otherwise back to Phase 1;

server: receive **Ph2**(Φ).
Compute $\Phi^* = \bar{H}(\Phi)$. Check a queue is bound with Φ^*. If no queue, NACK. Otherwise scan queue from start for first item matching $(M, \bar{H}(\Phi||M))$.
If not found, NACK. Otherwise, M is authentic; delete queue, send ACK and process M immediately.

3.2 A blind RSA signature scheme with public micro-metadata

Let us first sketch the steps between the Requester and the Signer taking, for simplicity, a concrete example of four levels of reputation: 0 to 3.

Let $n = pq$ be an RSA modulus of length L bits, where $p - 1$, $q - 1$ are chosen to be coprime with the first nine primes greater than two: 3, 5, 7, 11, 13, 17, 19, 23 and 29. We denote this sequence as g_i, $i \in [9]$. Let λ be the LCM of $p - 1$ and $q - 1$. The value of i represents public microdata. A coupon requires two reputations, current and new, and for the three level system with an oblivious 0 we have nine microdata values:

old	new	μdata, $\epsilon(a, a')$
0	0	1
0	1	2
1	0	1
1	1	3
1	2	4
2	0	1
2	1	5
2	2	6
2	3	7
3	0	1
3	2	8
3	3	9

Here we assume that a single sensing report cannot cause a reputation difference larger than 1, except the outcome 0 is always possible. However, if the new reputation is 0, it does not matter what the old reputation was since such a coupon has no value for an attacker. The above table defines the function $\epsilon(a, a')$ that encodes the reputations into a single microdata value. Also define the product exponent

$$t(a) = \prod_{a'} g_{\epsilon(a,a')}$$

Assume that M is the message to be signed blindly and that $H(x)$ is a Full Domain Hash (FDH), i.e., a cryptographic hash function mapping an arbitrary message on an L-bit string.

1. The Requester chooses two L-bit strings $b_{1,2} < n$ at random and sends to the Signer a 3-tuple that contains

$$\begin{aligned} &R, \\ &B = H(M) b_1^{t(a)} \bmod n, \\ &W = b_1 b_2 \bmod n, \end{aligned}$$

where R is the sensing report.

2. The Signer knows the current reputation a of the Requester from its certificate and it calculates the new reputation a' based on the report. Next it computes

$$\begin{aligned} e &= g_{\epsilon(a,a')} \\ d &= e^{-1} \bmod \lambda \\ s_1 &= B^d \bmod n. \\ s_2 &= 1/W \bmod n \end{aligned} \tag{1}$$

and returns the 3-tuple (a', s_1, s_2) to the Requester. Note that d can be pre-computed for all nine possible e.

3. The Requester calculates

$$e = g_{\epsilon(a,a')}$$

and determines

$$B' = s_1^e \bmod n\,.$$

Notice that

$$B' \equiv s_1^e \equiv B^{de} \equiv B \mod n\,.$$

The Requester checks that $B' \equiv B$ and if this holds, then the Requester also verifies that

$$W s_2 \equiv 1 \mod n\,.$$

and computes

$$W' = (s_2 b_2)^{t(a)/e} \bmod n\,.$$

Now notice that

$$W' \equiv 1/b_1^{t(a)/e} \mod n$$

Given that

$$s_1 \equiv B^d \equiv H(M)^d b_1^{t(a)ed/e} \equiv H(M)^d b_1^{t(a)/e} \mod n\,,$$

the Requester computes the unblinded signature of M as

$$s = s_1 W' \equiv H(M)^d \mod n$$

for public metadata (a, a').

The signature can be publicly validated by checking that

$$H(M) \equiv s^{g_{\epsilon(a,a')}} \mod n\,.$$

Now recall that the reputation *certificates* (as opposed to coupons) only use one kind of reputation: current. In our three-level reputation system, the mapping of reputation on metadata is straightforward:

reputation	μdata $\epsilon(a)$
0	1
1	2
2	3
3	4

with ϵ only having one argument a and $\{g_i\} = 3, 5, 7, 11$. Here reputation 0, although worthless, is still certified to prevent incoming reports from unregistered users. The Registration Authority keeps the user's current reputation, but can alter it: reduce, if no coupon has been received over a long period of time, or boost, if the client brings a large number of coupons with the same recommended reputation. It can also deny the reputation boost in the coupon or ignore lapses of reputation based on the historical record it keeps in the context σ. There is no logical reason that such adjustments should be more than by 1, since they are supposed to be cumulative and since there are only three significant reputation levels anyway. The only exception is that the reputation can be dropped to 0 from any level if an attack is detected.

Limiting the maximum variation to one unit otherwise, we arrive at the following function $\hat{t}(a)$ for blinding the certificate:

reputation	$\hat{t}(a)$
0	$g_{\epsilon(0)}$
1	$g_{\epsilon(1)} g_{\epsilon(0)} g_{\epsilon(2)}$
2	$g_{\epsilon(2)} g_{\epsilon(1)} g_{\epsilon(3)}$
3	$g_{\epsilon(3)} g_{\epsilon(2)}$

which should be used instead of $t(a)$ with the above three-step signing procedure.

3.2.1 Security analysis and efficiency

The security of the above solution is based on the analysis presented in (Amjad et al., 2025). A major difference between that work and ours is in the fact that the public metadata associated with the signature in the cited work is a matter of agreement between the parties whereas in our case it is solely at the discretion of the signer. Our method, which is based on the same RSA construction, avoids having to receive the metadata before submitting the blinded document for signature. Also, since the metadata is not up to the Requester, there is no point in including it in the document hash: our security requirements do not protect the Requester from the Signer's arbitrary choice of the new reputation; the Requester is not involved in the process.

However, in both the cited work and the present proposal, the metadata selects the exponent pair (e', d'), where $e'd' \bmod \lambda = 1$, used in the encryption/decryption of a specific message with which the metadata is to be associated. The key circumstance that limits the set E of allowable e' values is the so-called algebraic constraint:

$$(\forall k, p_1|e_1, \ldots, p_k|e_k, e_1, \ldots, e_k \in E) \prod_{i=1}^{k} p_i \notin E \tag{2}$$

where all p_i are prime and $k = |E|$. The constraint thwarts the anonymous signing attack (Borisov's attack, see ref 16 in (Amjad et al., 2025)) whereby the adversary chooses some message M_0, finds $M_0^{e_1/p_1}$ and supplies the report that is likely to cause the choice of the exponent e_1. After blind-signing and unblinding the adversary receives

$$M_1 = M_0^{e_1 d_1/p_1} \bmod n\,.$$

The attacker then computes $M_1^{e_2/p_2}$ and has it blind-signed after sending the report that is likely to select exponent e_2, getting

$$M_2 = M_0^{e_1 e_2 d_1 d_2/(p_1 p_2)} \bmod n\,.$$

By continuing the process, the adversary eventually obtains

$$M_k = M_0^{e_1 \ldots e_k d_1 \ldots d_k/(p_1 \ldots p_k)} \bmod n$$

If set E does **not** satisfy the constraint given by Eq 2, then the exponent $e = p_1 \ldots p_k$ may be in it, and if it is, M_k is a successful forged signature of M_0. Indeed,

$$M_k^e \equiv M_0 \mod n$$

In the light of the above, the exponent selection function ϵ should have a range that is compatible with the algebraic constraint 2. In the seminal work (Abe and Fujisaki, 1996) this was achieved by using a cryptographic hash function for ϵ, and the analysis found in (Gennaro et al., 1999) indicates that such a hash function would need a very large range (a 512-bit output for a 1024-bit RSA modulus) to satisfy condition 2 with a high probability. The authors of (Amjad et al., 2025) proposed to implement the exponent function as $H(\sigma||\ldots)$ where σ is static salt, which is a number selected at random and then validated through constraint 2 for all possible values of metadata (and if necessary re-selected and re-validated repeatedly).

Neither of these methods work for us as we are limited to the IoT power and energy budget; we wish to avoid costly full-range modular exponentiation. In the light of the cited work we defined our exponent function $g_{\epsilon(\ldots)}$ as one having a prime-number output which trivially satisfies the algebraic constraint: none of the prime numbers is a product of primes. While clearly too expensive in the general case of a large $|E|$, it is perfectly adequate for our 3-level reputation system, while only requiring at most 7 modular multiplications for signature validation (s_1^{29}) and up to 19 ones for blinding and up to 17 ones for unblinding:

$$b_1^{3\times13\times17\times19},\ (s_2 b_2)^{13\times17\times19}$$

which are well within the capabilities of a low-power IoT platform. The cost can be nearly halved if the platform possesses a modest amount of flash memory, since $b_1^{t(a)}$ can be precomputed for all a and several thousand random b_1, and stored in a non-volatile memory before the device is deployed in the field. Those values can then be used once until the device is docked and can spend more time/energy to prepare the next portion.

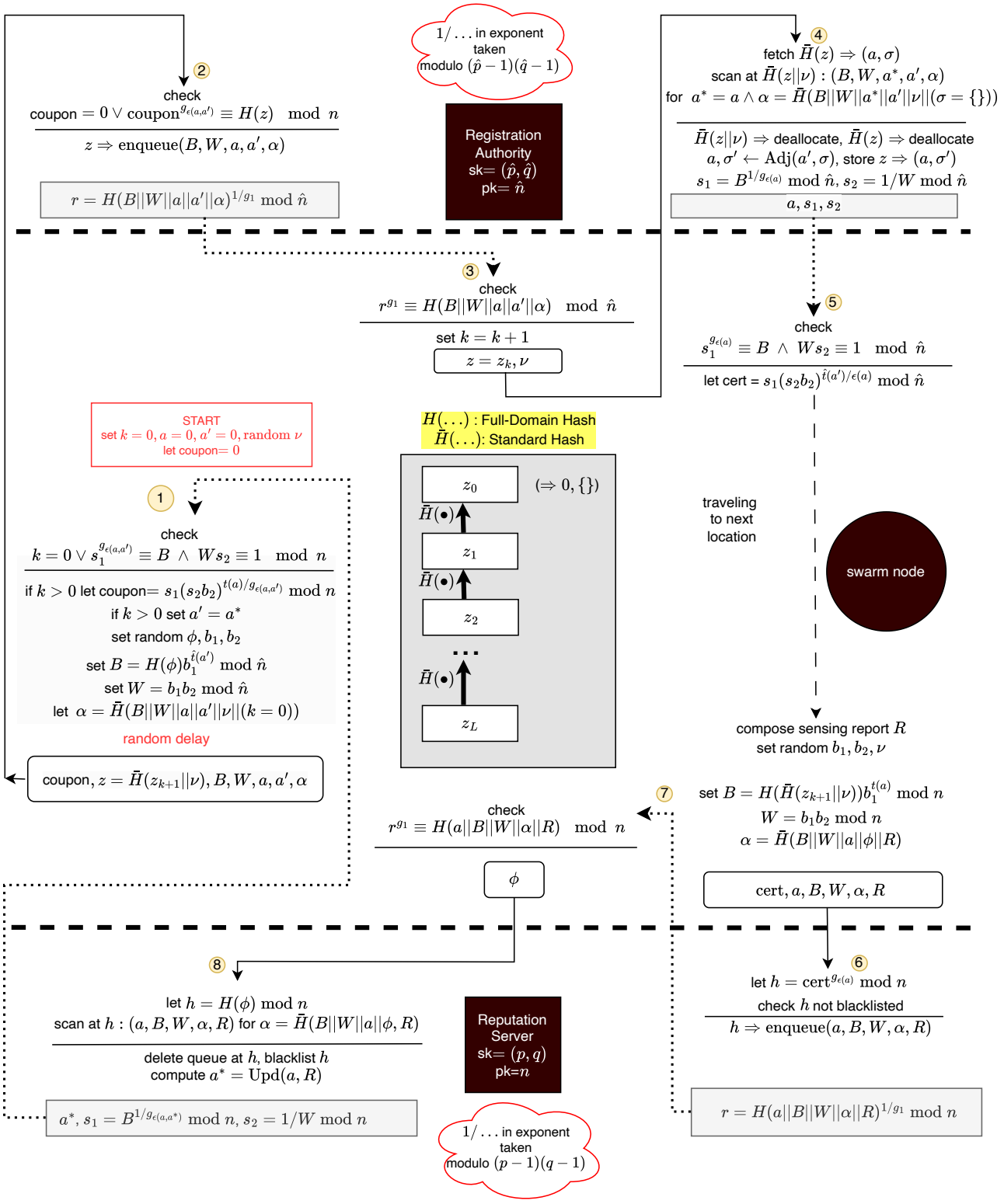

Figure 1: Anonymous reputation update: protocol

4 Bringing all pieces together

A sketch of the Anonymous Reputation Update Protocol (ARUP) that displays all required communication formats and cryptographic computations is shown in figure 1. The protocol goes through a sequence of rounds, each of which consists of 8 steps. In this section we will discuss all the steps in detail, but first let us focus on the general arrangements.

Each step of the protocol starts with a principal receiving a transmission as a fixed-format tuple, which is then validated. If the content received proves invalid, then the reaction to this depends on the category of the principal:

- A **server** just ignores the received content assuming that there was either a transmission/reception error or a deliberate attack. No action is taken.
- A **client** upon detection of an error backs up two steps. For step 1, two steps back brings the client to Step 7.

When all checks have been completed, the principal performs some security computations (and in steps 4,5 and 8 also some application-specific processing) and transmits a fixed-format tuple to complete the step.

Note that the servers are not aware of the protocol step for any given client. The client sends a step-related message invoking a particular *service* on the server. According to section 3.1.1 step 2 is Phase 1 of the Guy Fawkes protocol and step 4 is Phase 2, server-wise. Ditto for steps 6 and 8, respectively. The servers do their Phase 1/2 work with any number of clients intervening between the phases with their own Phase 1/2 requests.

RSA parameters. Denote the length of the modulus in bits $\mathscr{L}$, which would typically but not necessarily be 2048. For the RA, set random primes $\hat{p}$ and $\hat{q}$, each $\mathscr{L}/2$ bits in size, such that none of the first nine odd primes divides $(\hat{p}-1)(\hat{q}-1)$. Set RA's secret key sk=$(\hat{p},\hat{q})$ and public key $\hat{n}=\hat{p}\hat{q}$. Ditto for the RS parameters p,q,n and the corresponding secret key. For compactness we will use the abbreviation $1/g_x$ in the exponent to stand for

$$g_x^{-1} \bmod (\hat{p}-1)(\hat{q}-1)\,,$$

i.e. the RA's private exponent, and similar for the RS with p substituted for $\hat{p}$ and q for $\hat{q}$. We use the notation $H(x)$ for the full-domain cryptographic hash of sufficient security (e.g. 256 bits). This function returns an $\mathscr{L}$-bit number on an argument of any bit-size. Functions g_i, $\epsilon(a)$ and $\epsilon(a,a')$ are defined by table in section 3.2.

Transmissions and bit string length. Each transmission is a tuple of bit strings of certain length. The protocol uses four standard lengths:

L: the length of the RSA modulus, $\mathscr{L}$, typically 2K bits or 256 bytes;
M: the standard hash length, typically 256bits or 32 bytes;
S: a small integer, 8 bits or 1 byte; and finally
B: a blob of arbitrary size, used for signal reports and historical contexts.

State handling. Each server makes use of a KVS to keep its internal data:

The **RS** uses its KVS for maintaining two-phase Guy Fawkes transactions and storing its transmissions against input messages (to ensure cheap retransmission). Additionally, the RS maintains the black list of certificates to prevent their repeated use. The **RA**'s KVS is used for two-phase Guy Fawkes transactions as well as for keeping current reputation levels and contexts for all anonymous users. Coupons need not be blacklisted since they refer to Winternitz chain points, which by their nature cannot be reused.

The **client** stores various items in its private storage (which does not have to be a KVS).

- its current round k of the protocol. Initially $k=0$;
- private M-bit key z_L, where $L>1$, set at random; its Winternitz chain $\{z_k\}$, such that for all $i\in[L]$, $z_{i-1}=\bar{H}(z_i)$. The value of z_0 must match the GK ticket issued on behalf of the client's owner. The function $\bar{H}(x)$ is a standard cryptographic hash, e.g. SHA256;
- its latest transmission (for possible retransmission);

- client's round variables:
  size B: R
  size L: b_1, b_2, B, W;
  size M: α, ν, ϕ
  size S: a, a'. Initially $a = 0$.

In the sequel, temporary variables with a lifetime of a single step are introduced by 'let', while assignments to round variables are notated by the keyword 'set'.

4.1 Steps of the protocol

Step 1 (CLIENT)

Receive: if $k > 0$ then (a^*, s_1, s_2), format: (SLL); else nothing

Verify: $k = 0$; otherwise $s_1^{g_{\epsilon(a,a^*)}} \equiv B, W s_2 \equiv 1 \mod n$.

Execute: let coupon $= \begin{cases} s_1(s_2 b_2)^{t(a)/g_{\epsilon(a,a^*)}} \bmod n & \text{if } k > 0 \\ 0 & \text{otherwise} \end{cases}$

set $a' = \begin{cases} a^* & \text{if } k > 0 \\ 0 & \text{otherwise} \end{cases}$.
set random ϕ, b_1, b_2.
if $k = 0$ set random ν.
set $B = H(\phi) b_1^{\hat{t}(a')} \bmod \hat{n}$; $W = b_1 b_2 \bmod \hat{n}$.
set $\alpha = \bar{H}(B||W||a||a'|\nu||(1, \textbf{if } k = 0; \textbf{else } 0))$.

Transmit: $(\text{coupon}, \bar{H}(z_{k+1}||\nu), B, W, a, a', \alpha)$ format: (LMLLSSM)

Comment: If validation of condition $W s_2 \equiv 1 \bmod n$ fails, this does not have to cause step failure and retransmission. If the node has the resources to compute full exponentiation it can resolve the condition with respect to s_2 by itself and complete the step.

Step 2 (RA)

Receive: $(\text{coupon}, z, B, W, a, a', \alpha)$ format: (LMLLSSM)

Verify: $\text{coupon} = 0 \vee \text{coupon}^{g_{\epsilon(a,a')}} \equiv H(z) \mod n$.

Execute: enqueue (B, W, a, a', α) against z in KVS. Format (LLSSM)

Transmit: $H(B||W||a||a'||\alpha)^{1/g_1} \mod \hat{n}$ format: (L)

Comment: $1/\ldots$ in exponent is taken modulo $(\hat{p} - 1)(\hat{q} - 1)$.

Step 3 (CLIENT)

Receive: (r) format: (L)

Verify: $r^{g_1} \equiv H(B||W||a||a'||\alpha) \mod \hat{n}$

Execute: set $k \leftarrow k + 1$

Transmit: (z_k, ν) format: (MM)

Step 4 (RA)

Receive: (z, ν) format: (MM)

Verify:
KVS has a key-value pair $\bar{H}(z) \Rightarrow (a, \sigma)$,
with some a, σ [format (SB)];
KVS has a key-value pair $\bar{H}(z||\nu) \Rightarrow$ queue;
queue has an element (B, W, a^*, a', α) [format (LLSSM)],
where $a^* = a$ and $\alpha = \bar{H}(B||W||a^*||a'||\nu||(1, \textbf{if}\, \sigma = \{\}; \textbf{else}\, 0))$;
commit to first found.

Execute: deallocate key $\bar{H}(z)$ in KVS, deallocate key $\bar{H}(z||\nu)$ in KVS
$a, \sigma' \leftarrow \textbf{Adj}(a', \sigma)$
store in KVS: $z \Rightarrow (a, \sigma')$
$s_1 = B^{1/g_{\epsilon(a)}} \mod \hat{n}$, $s_2 = 1/W \mod \hat{n}$

Transmit: (a, s_1, s_2) format: (SLL)

Comment: $1/\ldots$ in exponent is taken modulo $(\hat{p} - 1)(\hat{q} - 1)$.

Step 5 (CLIENT)

Receive: (a^*, s_1, s_2) format: (SLL)

Verify: $s_1^{g_{\epsilon(a^*)}} \equiv B$, $Ws_2 \equiv 1 \mod \hat{n}$

Execute:
produce sensing report R
set $a = a^*$
let cert $= s_1(s_2 b_2)^{\hat{t}(a')/g_{\epsilon(a)}} \mod \hat{n}$
set random ν, b_1, b_2
set $B = H(\bar{H}(z_{k+1}||\nu))b_1^{t(a)} \mod n$
set $W = b_1 b_2 \mod n$
set $\alpha = \bar{H}(B||W||a||\phi||R)$

Transmit: $(\text{cert}, a, B, W, \alpha, R)$ format: (LSLLMB)

Step 6 (RS)

Receive: $(\text{cert}, a, B, W, \alpha, R)$ format: (LSLLMB)

Verify: let $h = \text{cert}^{g_{\epsilon(a)}} \bmod \hat{n}$; h not blacklisted

Execute: enqueue (a, B, W, α, R) against h in KVS.

Transmit: $(H(a||B||W||\alpha||R)^{1/g_1} \bmod n)$ format (L)

Comment: $1/\ldots$ in exponent taken modulo $(p-1)(q-1)$.

Step 7 (CLIENT)

Receive: (r) format (L)

Verify: $r^{g_1} \equiv H(a||B||W||\alpha||R) \mod n$

Execute: nothing

Transmit: (ϕ) format (M)

Step 8 (RS)

Receive: (ϕ) format (M)

Verify: let $h = H(\phi) \bmod \hat{n}$
KVS has a key-value pair $h \Rightarrow$ queue;
queue has an element (a, B, W, α, R) [format (SLLMB)],
where $\alpha = \bar{H}(B||W||a||\phi||R)$ commit to first found.

Execute: blacklist h, deallocate key h in KVS
$a^* = \mathbf{Upd}(a, R)$

Transmit:
$(a^*, B^{1/g_{\epsilon(a,a^*)}} \bmod n, 1/W \bmod n)$ format (SLL)

Comment: $1/\ldots$ in exponent taken modulo $(p-1)(q-1)$.

4.1.1 Gate Keeper: protocol for Onboarding/Offboarding

To launch ARUP for a new client the following steps are taken:

1. The client's owner obtains the GK public key $(\check{e}, \check{n})$ from an authenticated, trusted public server.
2. The client privately produces the Winternitz chain $\{z_i\}$ and blinds z_0 thus: $B = b^{\check{e}} H(z_0) \bmod \check{n}$, where b is a random number the same size as $\check{n}$
3. The client's owner submits to the GK their real-life credentials and the value of B using an authenticated channel (could also be protected, if the client's owner wishes to keep the membership in an ARUP system confidential and trusts the GK that it will keep it so). The GK checks the credentials and responds with the blind ticket $T^* = B^{\check{d}} \bmod \check{n}$, where $\check{d}$ is the secret exponent (a part of the GK private key).
4. The client unblinds the ticket: $T = T^* b^{-1} \bmod \check{n}$ and provides the value of T to the client's owner.
5. The client's owner prints (T, z_0) as a QR-code on a sheet of plain paper and posts it to the RA anonymously.
6. The RA scans the QR code and verifies that

$$H(z_0)^{\check{e}} \equiv T \mod \check{n}\,,$$

   If satisfied, the RA deploys $(0, \emptyset)$ against z_0 in its KVS. An anonymous client has been onboarded.

Offboarding is not always required, but could be essential if the client's engagement is time-bound. For instance, the GK could grant the user onboarding against a collateral on the promise of reaching a certain minimum level of reputation by a certain deadline at which the collateral would be refundable. If offboarding of this kind is desirable we propose the following modification of ARUP.

If the round is to be the client's last round, then in the course of **Step 1** above ϕ should be set as follows (rather than at random):

$$\phi = \text{AccNo}||\text{GKnonce}$$

making sure that the length of ϕ is not equal to M. Here AccNo is the number of the account that the user has with the Gate Keeper in their real name, and the value of the random GKnonce is refreshed and published by the GK every day.

Step 5 of the last round is the last step in the round and it is limited to input verification and computation of the certificate:

Step 5 **last round** (CLIENT)

Receive: (a^*, s_1, s_2) format: (SLL)

Verify: $s_1^{g_{\epsilon(a^*)}} \equiv B,\ W s_2 \equiv 1 \mod \hat{n}$

Execute:
set $a = a^*$
let cert $= s_1 (s_2 b_2)^{\hat{t}(a')/g_{\epsilon(a)}} \mod \hat{n}$

Transmit: None

Next the certificate (which was signed blindly by the RA with the current reputation as public metadata) is presented by the user to the GK along with the reputation a, AccNo and GKNonce. The GK checks the signature and refunds (all, part of or none depending on a, the date associated with GKnonce and the contract associated with AccNo).

4.2 Security analysis

The scheme presented in the last subsection seems quite intricate. Its security, however, is based on a small set of assumptions which follows below:

A1 Random Oracle model. We assume that both hash functions, the short one $\bar{H}(x)$ and the full-domain one $H(x)$, are random oracles, namely that they respond with a random number to every value of the argument. The response is the same if the same value of the argument is presented.

A2 Both hash functions are preimage resistant, and the shorter hash is additionally second-preimage resistant.

A3 RSA is secure enough to make modulus factorisation impractical over the lifetime of the public key. No forward secrecy is required of either the RA or the RS. The only secret either server keeps is its private key, and no certificate or coupon survives the revocation of the signing key used to sign it.

Step 1. If $k > 0$ then the blinded version of the coupon, s_1, and the inversion of the (blinded) blinding factor s_2 have been provided in a message alongside the RS-recommended new reputation a^*. The verification required is standard signature check, see Section 3.2, and the obvious inversion check. As a result a valid coupon is produced, which is

$$H(\bar{H}(z_{k+1}||\nu))^d \bmod n,$$

where d is the private RSA exponent for the public $g_{\epsilon(a^*,a)}$:

$$dg_{\epsilon(a^*,a)} \equiv 1 \mod (p-1)(q-1)$$

Only the client knows ν and z_k, and it can broadcast the preimage $\bar{H}(z_k||\nu)$ to claim the coupon without risk of theft.

Next the client creates a new pair B, W to obtain the current certificate from the RA. The certificate is based on a fully random nonce ϕ, which has no connection to the chain and so needs to be unique and will be blacklisted after use (Step 8). Reuse of ϕ does not profit the client as the resulting certificate would be rejected at Step 6. Finally, the authenticator α is computed to secure the output message, which also contains the coupon (or a zero-coupon, if $k = 0$), the coupon preimage $z = \bar{H}(z_k||\nu)$, the blinded certificate data B, W, and the old and new (recommended) reputations. The output message is sent after a random delay to time-decorrelate its sending with the receipt of the coupon. The delay time should be sufficient to make sufficiently many coupons issued by the RS potential candidates for the cause of the output at Step 1.

Step 2. First of all, if it receives a nonzero coupon the RA validates z, but if the coupon value is 0, no validation is necessary since it is a special case: a new client has just started. It has the initial reputation 0 requiring no update; more importantly, no certificate has been issued yet on behalf of this client and so no coupon needs to be consumed to fulfil security objective 1 presented in Section 2.

At this step the RA receives Step 1's output message as input. The RA has a simple job of issuing a receipt and putting the incoming message on the queue. To do the former, the RA uses the least public exponent available g_1 to make it cheaper for the client to validate the receipt. Since g_1 is also used in coupons and certificates, one might think that this is a security risk, but in fact it is not. Coupons and certificates blindly sign a single size-M bit string, namely they raise the hash of a size-M bit string to one of the secret exponents. Validation of the coupon or certificate (after appropriate unblinding) consists in presenting the original bit string of size M to be hashed and the resulting hash to be compared with the result of raising the coupon/certificate to the public exponent. According to A2, no bit string longer than M that produces the same value can realistically be found (a second preimage). It follows that the receipt, even though its hash argument is publicly known, cannot be transformed into a valid coupon or certificate.

The purpose of the receipt should be clear from Section 3.1.3: it is an effective countermeasure for the Guy Fawkes man-in-the-middle attack.

Step 3. At this point the client (the same client that sent the components receipted by the incoming message of Step 3) checks the receipt and satisfies itself that the data sent in Step 1 were correctly received by the RA. It does not matter if the RA first received a derivative variant of Step 1 output, i.e. modified

by the attacker or by a channel error. It is only important that the Step 1 message *has been* received intact . If the values z_{k+1} and ν are published now, the genuine Step 1 output will be the first on the queue *with the correct value of authenticator* α, since it is there *already* and no other principal knows the values z_{k+1} and ν *yet*.

Since the receipt is both genuine and correct, the RA will definitely produce a new certificate and so at this point the client enters its round $k+1$. To start the round the client releases z_{k+1} and ν from Step 5, or, if it is round 1, then, the client fetched a fresh ν from Step 1.

Step 4. The first validation step is to see if the z received is the next point on the chain z_k after the previous round, $z_{k-1} = \bar{H}(z)$. If it is, then the RA must hold in its KVS a key-value pair with z_{k-1} as the key and some (a, σ) as a reputation-context pair. Note that the success of this validation action does not yet guarantee that the originator of the Step 3 output message is genuine, it could be the "man in the middle": an agent that intercepted the client output and is now using it to get a certificate. The next validation action is what eliminates the man in the middle: the RA searches the queue associated with z, which was defined as $\bar{H}(z_{k+1}||\nu)$ back at Step 1 and which is exactly the value of $\bar{H}(z_k||\nu)$ in the current step since the client has entered the next round. The queue was established in Step 2 before issuing the receipt. This validation action checks that the queue exists and that a valid queue element is present. If it does and it is then the input to this step is from the genuine owner of the chain and the coupon has the correct old reputation.

Consequently, there is no further use for the queue and the reputation-context pair, and they are both deallocated. There is no need to blacklist the coupon, since its repeated use is impossible already: the chain point included in the coupon data would be rejected by the RA as its hash no longer matches a key in the RA's KVS. The RA uses the external function **Adj** that takes the proposed new reputation (which was included in the record found on the queue, and which, before that, was a^*, the output of external function **Upd** invoked by the RS) to the actual new reputation on the basis of the current context.

The fact that the old reputation of the submitter of a coupon is known to the RA and the fact that it is included in the coupon as well is crucial for preventing certificate trading. Indeed, since certificates are based on an unlinked nonce (ϕ), two colluding clients with different reputations could exchange their certificates (along with the nonces) to achieve an illegal reputation gain for one with a compensation to the other, which loses. This scenario is rendered completely ineffective by including the old reputation in the coupon, which would not match the RAs records if the certificates were swapped.

Step 5. Validation of the certificate follows the same procedure as coupon validation in Step 2. The unblinding follows and finally the client obtains the certificate in the form

$$H(\phi)^d \bmod \hat{n}\,,$$

where, as usual,

$$dg_{\epsilon(a)} \equiv 1 \mod (\hat{p}-1)(\hat{q}-1)\,.$$

The value of ϕ is a nonce produced in Step 1. Next, the client travels to a sensing location, which takes some time. The longer the time the weaker the temporal correlation between the issuance of the blinded certificate and the presentation of its unblinded version to the RS and the greater the chance that several such acts are in progress due to other clients' actions.

Eventually the client arrives at its next sensing location, produces a report, and a blinded version of the future coupon. The two-phase Guy Fawkes technique here is the same as that used in Step 1: the output record includes the certificate, the current reputation, the blinded coupon, the report and an authenticator which depends on yet undisclosed value of ϕ.

Step 6 is analogous to Step 2 except for the fact that ϕ is a random nonce (intentionally) unlinked with the client's persistent structures. This is necessary to unlink the client identity (even anonymised, even a temporary one) from the report to prevent deanonymisation by analysis of visited locations against time. A negative consequence of this is that certificates by themselves are eminently clonable and reusable. The situation is somewhat similar with electronic cash where the same coins could be spent repeatedly if it were not for spent coin registration.

We follow the idea of electronic cash and introduce the concept of *blacklisting* a certificate. Its contents $H(\phi)^d \bmod \hat{n}$ are first extracted by applying an appropriate public exponent $g_{\epsilon(a)}$ to obtain $H(\phi)$ and then, when the certificate is considered "spent", the value of $H(\phi)$ is added to the black list. This is done in the same step in which a (blinded) signed coupon is created to satisfy security objective 1 (Section

2). Note that both certificates and coupons use the lowest public exponent g_1 to minimise the cost of extraction. Blacklisting the original $H(\phi)^d \bmod \hat{n}$ would bring no additional benefit, since extraction is necessary to validate ϕ in the next step anyway.

Another security problem stems from the fact that there could potentially be more than one RS. The Guy Fawkes technique we use for piggybacking data on the authenticator α requires a distributed queue of records (one for each value of h), since the "man in the middle" can utilise nonlocal communication and run the protocol with a different RS in an attempt to illegally obtain a coupon for himself. While efficient solutions exist, for example Apache Kafka (Narkhede et al., 2017), it might be even more efficient to use one Cloud server (or several, partitioned by the key) for this Step and Step 8, while keeping the computation of the function **Upd** in the Fog. As we are primarily concerned with the security aspect here, we remark that no matter how the distributed queue is implemented (if at all), it must maintain strict consistency in real time to prevent man-in-the-middle attacks.

Step 7 is analogous to Step 3 in the validation part. Once the receipt has been validated, the client is reassured that its data for the coupon has been received intact and has been placed on the queue. It sends ϕ back to the server to initiate the second phase of piggybacking.

Step 8. The first validation action is to compute the key of the queue $h = H(\phi)$, which should be the same value as h in Step 6 and so a queue must exist against it in the server's KVS. The second verification step is to fetch the item on the queue that passes validation by α given the now acquired value of ϕ, similar to the queue scan in Step 4, except this time only one condition is being checked.

When verification is successfully completed and, as a side effect, the valid queue item is found, then it contains the current reputation a certified by the fully verified certificate. So the value of a is secure and the value of R is also secure due to the authenticator α which depends on the certificate validator ϕ. The RS is ready for a secure reputation update, but first it is time to blacklist the certificate and remove the queue from the KVS. Finally the blinded coupon is signed and transmitted for the client to receive in Step 1.

Onboarding The security of onboarding is obvious from the protocol construction. The GK cannot signal to the RA any of the client's owner credentials since it signs the client's z_0 blindly and since z_0 is the only source of user identity for the RA.

Offboarding Clearly if the length of ϕ in the last round is different from M, the certificate cannot be used for reputation update, so need not be blacklisted. This is essential, since otherwise the GK and RS would have to address the consistency problem that arises when the same certificate is used for offboarding and to obtain a coupon within a short interval of time. For obvious reasons this must never succeed, and it will not, due to the fact that the RS would only accept a size-M ϕ and that no size-M ϕ can be found which produces the same hash image as a different (size) bit string, due to assumption A2 (Section 4.2).

Furthermore, the client will not be able to claim that the reputation was reached on a later day because of the nonce incorporated in the value of ϕ. This is due to the fact that the nonce value cannot be predicted and has to be obtained *on the day*, which means that the reputation value claimed by the offboarding client was not obtained *earlier* than that day. This prevents the client from stopping to use the system as soon as the desired reputation is reached (so that the user may recover their collateral rather than having to risk reputation loss by further (in)activity).

4.3 Post-quantum threats

The security of ARUP depends on the assumption that the (public) knowledge of the RSA public key does not lead to the discovery of the private key via some feasible computation. It is well known that this assumption does not hold in the current, post-quantum reality. That is why nations have plans to replace their classical public-key cryptography by some quantum-resistant algorithms, which tend to be power hungry and with much longer signatures. This may suggest that the lifetime of our proposal is rather limited by the transition date (set by various nations around 2035), but that is not the case for the following reasons:

1. There is nothing in the ARUP that requires forward secrecy. RSA is only used for signing and the signatures are only verified there and then, and if found valid are discarded never to be examined

again. An attacker gains nothing from storing a cyphertext for the technology to catch up to break it.

2. The pseudonym identification method is based on Guy Fawkes protocols, and those are clearly post-quantum, since they are based on the properties of a hash function, rather than any algebraic operation. While hash-function pre-imaging can be accelerated by a quantum computer executing Grover's algorithm, the quantum security parameter is the square root of the image length, which is sufficient for any practical purpose.

3. RSA keys can be rotated frequently at negligible cost to the client. That limits the time in which the attacker may achieve its objectives. Even when RSA is broken by a CRQC, one needs to consider the time and the energy cost of a single successful attack. Also, one must examine the potential damage such an attack can inflict.

Continuing with point 3 above, we note that although paper (Gidney and Ekerå, 2021) established a baseline of 20 million physical qubits for RSA-2048, the Gidney 2025 optimisation (Gidney, 2025) managed to reduce this number to approximately 900,000 physical qubits at an error rate of 10^{-3}. It can be calculated that transition to RSA-4096, which is still good for our purposes, see Section 6.1, would require up to 3 mln qubits. However, the optimised computation will be much longer, about 150 hours (around 6 days), compared to the 8 hours for the unoptimised method in (Gidney and Ekerå, 2021). If we consider the amount of *energy* spent, which is proportional to the product of the qubit number and processing time, we will see that it has barely changed between these predictions spanning 4 years of active development. According to the RAND corporation 2023 report (Welser et al., 2023), the power consumption (dominated by control electronics for large systems) is approximately 6W per physical qubit. For a 3mln physical qubit system operating over more than 150 hours (when the modulus is 4096-bit, not 2048-bit) the energy burn is measured in GigaWatt-hours. One GWh costs hundreds of thousands of dollars which is how much an attack on one key would cost. And if a key is rotated, say, every four days, that money would be wasted, since the private key learned that way would expire before its calculation is complete.

So for our chosen application, namely the anonymous reputation update, RSA vulnerabilities are not a major threat. Three forms of attack can be foreseen:

Client-side: The client or a group of clients compute the RA private key. They can now create a number of fake top-reputation certificates to submit fake reports alongside them, persuading the RS to accept a false situation (e.g. road congested in some direction, traffic lights are required to turn green to remedy) to benefit the client passage, but this advantage will cease as soon as the RA public key is revoked and replaced.

Server-side: An external attacker computes the RA private key. It can now insert itself between the client and RA and execute a man-in-the-middle attack on the Guy Fawkes sequence, producing a correct Step 2 output without the genuine RA having received the Step1 output message. As a result, the attacker is able to submit another client's coupon on behalf of the victim client. This would set the wrong proposed reputation for the victim client and invalidate the second client's coupon. The attacker can also crack the RS private key (doubling the cost of the attack) and create fake coupons to invalidate the victim's coupon in a similar way.

None of the above seems to justify the enormous cost of the hack, and as soon as the affected keys are revoked and replaced and the affected clients restart with reputation 0, the system will heal itself.

A more significant threat is the possibility of an attack on the key distribution protocol which has to exist to rotate the RA and RS public keys. Authentication of those keys should be done by the GK as that is the only party the client's owner has to trust. If the GK were to run a pre-quantum key distribution protocol, an unacceptable long-term exposure would result, putting all operations at risk. Fortunately, a Guy-Fawkes solution exists with a negligible cost to the client.

Key distribution and rotation. The GK prepares a Winternitz chain for key distribution D_i of length l as follows:

$$\begin{aligned} D_l &= P_D; \\ D_i &= \bar{H}(D_{i+1}), \text{ for } i = 1 \dots l-1\,, \end{aligned}$$

where P_D is a private random value. All D_i as well as P_D are length M. The chain points $D_1, D_2 \ldots$ are published at regular intervals of a duration Δ, shorter than the reliably pessimistic estimation of the CRQC code breaking time. The distribution starts at T_0 when the first chain point, D_0 is published on an open, unauthenticated channel. All other points D_i are published near the times $T_0 + i\Delta$, also on an open, unauthenticated channel.

The values of $\bar{H}(D_0)$, T_0, and Δ are deployed in advance on an authenticated public sever, where the GK's public key is also deployed, so during the onboarding procedure this information becomes available to the client. The values of D_i are chain points and so they are authenticated by the client itself at a cost of one hash calculation. Before the system starts l key pairs are generated for the RA and another l for the RS and the following table is deployed in the clear on an authenticated server:

$$n_i \oplus H(D_i),\ \hat{n}_i \oplus H(D_i) \text{ for } i = 0 \ldots l\,.$$

Notice the use of the full-domain hash. The client can simply authenticate and download a copy of the table as part of its onboarding.

The security properties of this protocol are quite obvious. The Winternitz chain used here is quantum resistant and should ensure that the hash image of length M is long enough for a CRQC running Grover's algorithm to take reliably longer than Δ to pre-image. This condition is easily satisfied by the SHA512, so the public keys n_i, $\hat{n}_i$ will be known to the attacker at $T_0 + i\Delta$ at the earliest, and at time $T_0 + (i+1)\Delta$ they will be revoked. All actors should allow both the new and old keys near the rotation point to accommodate clock inaccuracy; this does not present a significant additional risk.

5 Optimisations

The above scheme can be improved without disturbing any of its fundamentals. We will list here two significant improvements motivated by (i) the desire to reduce the IoT platform's storage requirements and (ii) defending the servers against a possible DoS attack by a malicious client(s). The former may not be required if the IoT platform has sufficient flash memory or similar. The latter can be achieved by the networking infrastructure if it has defences against excessive submission, perhaps ones based on traffic analysis. Also DDOS can be prevented by GK's robust user-verification that reduces the possibility of a Sybil attack.

5.1 Storage for a Winternitz chain

In principle, there is nothing particularly challenging in finding sufficient storage for the chain on an IoT device. A million-strong chain, which would support reputation update at a rate 100 times a day for more than 27 years, while only requiring 32 Mb of read-only storage is available off the shelf. However for tiny mobile sensors with a small ROM/Flash it is possible to store a small proportion of the chain and yet provide access to every chain point at the cost of one hash recalculation (a few microseconds). This requires a small size RAM in addition to the ROM. Given the ROM S of size N and a RAM buffer B of size G (both sizes in units of hash length), we store a length $(N+1)G$ chain thus:

$$S_i = z_{iG}\,,$$

where $i = 1 \ldots N$, and we also fill in the RAM buffer with values

$$B_{G-1} = \bar{H}(S_1);\ \ B(j) = \bar{H}(B_{j+1}) \text{ for } j = G-2 \ldots 0$$

When all G hashes in the buffer have been used, we refill it as above substituting S_2 for S_1 to obtain the next G chain-points, etc.

In practice with a 32K byte RAM buffer and a 32K bytes ROM we have more than a million chain points available at a cost of 1 extra hash per chain point. Of course there is an additional penalty in latency when the buffer has been emptied; it cannot be used until filled in again, a delay of 1K times the hash calculation time. This is, however, completely benign as the protocol itself requires down time (much longer than the few millisecond needed to fill in the buffer) for event decorrelation.

5.2 Low-cost signature for RA/RS receipts

Another, much more significant source of inefficiency is found in Steps 2 and 6 of the protocol. The client needs an assurance that the server, rather than a man in the middle shadowing the server, has received

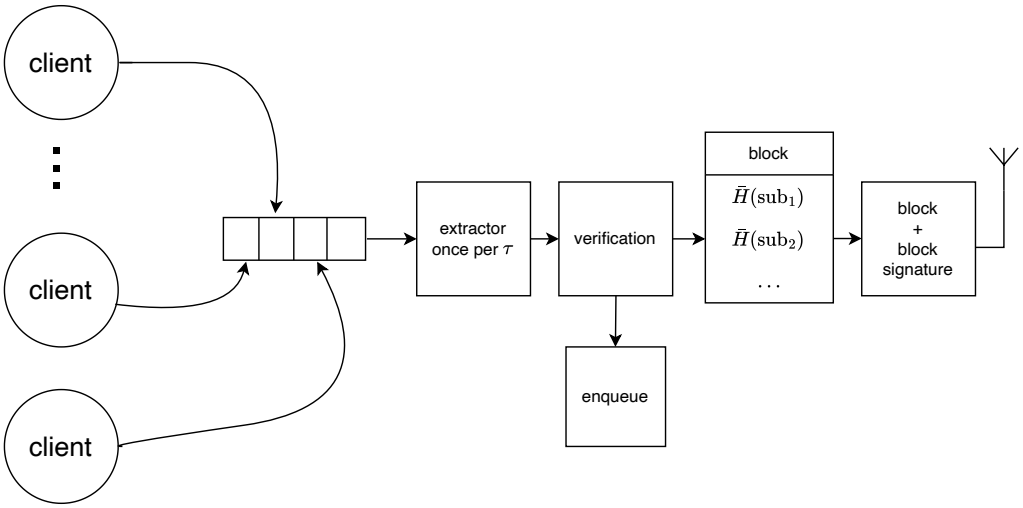

Figure 2: Optimisation of signing the receipt of Step 2 and 6 input messages

the message intact. It needs that assurance to send the Step 3/7 message, respectively, and no forward secrecy is required. That is, if the man in the middle forges this signature in a few minutes and tries to elicit the Step 3/7 message for intercepting and adapting it for the attack, it is likely to be too late, since the client will register a time-out and will change its Step 1 or Step 5 message before retransmitting. A small nonce to be added to the output messages of those steps is all it takes. So the attacker's only chance is to forge the server's signature quickly, in a matter of seconds. After that, the receipt signature (but not the private key) could be cracked by the attacker causing no harm, as the preimage withheld pending the receipt would have been published and made ineffective for repeated use.

For that kind of signature a full RSA signature algorithm is an overkill with its significant forward secrecy. Below we will propose a much weaker signature, yet sufficient to keep the attacker busy for some hours or at least minutes. The DDOS opportunity is to do with the fact that clients can keep sending random Step 1 and/or Step 5 messages which could only fail at further steps, but which would make the server issue receipts, i.e. to calculate a full RSA signature, which involves full modular exponentiation. For a 2K modulus (the minimum recommended length at present time), this would necessitate circa 3K modular multiplications. This may not be too many for a single event, but without throttling the submission rate exponentiation could saturate the server, especially a Fog one, which we assume that the RS is likely to be.

To propose a solution we first observe that DoS would be impossible if the server established a fixed maximum signature rate, which corresponds to its capacity to sign messages of a certain size. This would shift the attack back to the communication infrastructure, which already has this vulnerability, e.g. it is vulnerable to RF jamming. On the other hand, a legitimate client does not require instantaneous response to its situation report submission or a coupon redemption request, since a new report would not be ready very soon in networking terms. Our solution follows the idea in (Gu et al., 2009): combine submissions for signature and let each submitter verify that the signature signs their contribution among others, see figure 2.

Specifically, the server receives submissions for signature (Step 2 or 6 input messages) and stores them in a message buffer or ignores them if the buffer is full. Every τ seconds, an extractor process extracts all messages from the buffer and passes them on to the verification stage (which has some work to do in Step 2 but none in Step 6). The messages that did not verify are ignored and the rest are passed on to be put on a KVS queue in accordance with the protocol. At the same time the messages that have passed verification are digested using the short hash ($\bar{H}(x)$) and their hashes are collected into a single block in an arbitrary order. The maximum block length corresponds to the maximum buffer capacity. Finally, the block content J and the RSA signature $r = H(J)^{d^*} \bmod n^*$ are broadcast to the clients. Here for the RS: $d^* = d$ and $n^* = n$ and for the RA $d^* = \hat{d}$, $n^* = \hat{n}$, i.e. the private exponent and modulus, respectively.

When a client receives the block and its signature (J, r), it first of all checks if the short hash of its

Op	Bits	Exponent	Avg (ms)	Stddev (%)	Min (ms)	Max (ms)
ModMult	2048	–	0.1	0.88	0.10	0.11
ModExp	”	small (19635)	1.4	0.89	1.34	1.38
ModExp	”	full-domain	158.7	0.11	158.39	158.89
ModMult	4096	–	0.3	0.24	0.29	0.30
ModExp	”	small (19635)	3.9	0.57	3.87	3.97
ModExp	”	full-domain	943.3	1.79	928.79	979.83

Table 1: ESP32 performance data for modular multiplication and exponentiation using relevant parameters.

Table 2: SHA256 and Full-Domain Hash timings (total time in ms).

Length (bytes)	SHA256	FDH2048 (SHA512×4)	FDH4096 (SHA512×8)
32	0.05	0.25	0.50
64	0.07	0.25	0.50
128	0.08	0.40	0.80
256	0.09	0.45	0.89
512	0.13	0.54	1.08
1024	0.19	0.73	1.46
2048	0.32	1.11	2.22
4096	0.58	1.87	3.74
8192	1.10	3.39	6.77
16384	2.13	6.42	12.85

submission is included in the block. Then it checks the signature in the usual way:

$$H(J) \equiv r^e \mod n^* .$$

If the correct short hash does not occur in J or if the value of r does not validate J, the client attempts to submit in the next τ-interval.

The security of the batch signature is the same as the security of the ordinary RSA signature thanks to assumption A2 of our security analysis (Section 4.2), while the number of signatures per second is fully determined by τ and not at all by the submission rate. Note that the throughput of our solution is C_b/τ signatures per second, where C_b is the buffer capacity in messages, i.e. can be made as large as possible by increasing the buffer size. The trade off is between that and the latency, which is approximately $\tau + \tau_s$, where τ_s is the processing time for signing. One might object that that time also depends on the number of hash calculations and that is proportional to C_b, but in reality the processing time is dominated by exponentiation and the short hash calculation is orders of magnitude faster, so the latency depends on C_b very little. All these considerations lead us to the conclusion that should the signing process prove a limiting factor when under attack, i.e. should the speed of the server-side RSA signature calculation prove insufficient, the batch signature method could be a simple and effective remedy.

6 Experimental evaluation and reference implementation

As an example platform for an embedded implementation we chose an inexpensive and broadly available microcontroller ESP32 from Espressif Systems (Espressif Corp, 2025). It has a hardware accelerator for cryptography, which makes it quite suitable for our purposes. The accelerator handles RSA block sizes up to and including 4K bits, which makes it future-proof for at least the next 10 years.

Even though a wealth of performance data from the manufacturer is available, it is always wise to treat it with caution and perform one's own experiments to evaluate the timing of the protocol's building blocks with specific parameters to obtain figures one can rely upon in design arguments. We did just that and have made the code available on github to enable further scrutiny if required (comqas, 2026b).

Only the costs of the proposed *client* Steps (1,3,5 and 7) are important since the rest of the steps are executed on a server. One glance at the pseudocode (see section 4.1) is enough to establish that the client's execution time is dominated by the following algorithms:

- Modular multiplication
- Modular exponentiation

Step	Mult	Exp	Hash	FDH
1	5	3	2L+M, 2M	M
3	0	1	–	2L+M
5	5	3	2M, 2L+M+B	M
7	0	1	–	2L+M+B

Table 3: Complexity parameters of the ARUP protocol. For the hashes, the length of the argument is listed

- Hash
- Full-Domain Hash (FDH)

For modular multiplication and exponentiation performance data we time the full hardware-accelerator invocation path as used in ESP-IDF (context setup, operand loading, accelerator start, completion polling, and result read-back), not just the raw Montgomery core latency. The benchmarks run in a single task without overlap or hand-tuned assembly tricks, so the figures are conservative and representative of practical embedded use. For hashes we measure end-to-end timing using the standard ESP-IDF SHA API, so the reported values include all software and driver overheads (initialisation, locking, padding, and result read-back), not just the bare hardware engine latency. This also yields conservative, practical timings that reflect real application use rather than idealised per-block throughput.

The results are presented in Tables 1 and 2 for the arithmetic and hash algorithms, respectively. To illustrate the efficiency of our method compared to any other based on blind signature with public metadata (Amjad et al., 2025), where the metadata is used to produce a full-domain exponent using a hash function with special properties, we include in Table 1 contrasting data for full domain exponentiation, which is two orders of magnitude more time to execute. This makes it clear that our version with small factor exponents that defeats Borisov's attack has a significant advantage. Another pertinent observation comes from Table 2: we note that the execution time grows slightly less than linearly with the growth of the argument length, which makes it easy to provide an upper bound at any point of interest between the tabulated values.

Let us fully quantify the performance of the client's Steps. The glue code, which disassembles input records, feeds data into the four algorithms above and combines their results to form an output message according to the pseudocode definitions in section 4.1 introduces a negligible overhead on a processor with 240Mhz clock rate, such as the ESP32. Indeed Step execution is linear (no loops) and the glue code only moves data (a few hundred bytes) in memory. This would amount to no more than hundreds of microseconds and would hardly be visible or measurable on the millisecond scale that the basic algorithms are placed on. We chose to simply count the number of application of the algorithms per step and to note the size of hash arguments. The results are summarised in Table 3. It is evident that one round of the protocol, which combines the four client step above works out at 10 modular multiplications and 5 modular exponentiations. There are 4 invocations of the standard hash and 4 more of the full-domain hash. The cost of the arithmetic is 8ms per round for RSA 2K, and circa 23ms for RSA 4K.

The cost of the hash function invocations depends on the size of the report. For typical applications in traffic management, a report is no more than a few hundred bytes, possibly much less, as it contains information about observed traffic density, speed or similar. If we assume for an estimate that B=200, we get the following figures:

Size	Type	Instances	Bytes (RSA 2K)	Bytes (RSA 4K)	ms (RSA 2K)	ms (RSA 4K)
M	FDH	2	32	32	< 0.3	0.5
2M	Hash	2	64	64	negligible	negligible
2L+M	FDH	1	544	1056	< 1	< 2
2L+M	Hash	1	544	1056	< 0.2	< 0.2
2L+M+B	FDH	1	744	1256	< 1	< 2
2L+M+B	Hash	1	744	1256	negligible	negligible
				TOTAL	$< \mathbf{3}$	$< \mathbf{5}$

To conclude, a round of protocol on the client takes less than $8 + 3 = 11$ ms for RSA 2K (the current recommended minimum size) and less than $23 + 5 = 28$ ms for RSA 4K, which is the NIST recommended size until the year 2035 and possibly beyond. We stress that those performance figures are

quite conservative, since the handling of the RSA accelerator on the ESP32 has not been optimised and since the ESP32 is not necessarily the fastest microcontroller available today, but it is one of the cheapest and is broadly used.

6.1 Practical implementation: a sealed key-fob anonymiser

Performance of cryptographic algorithms affects embedded systems and IoT platforms in two ways. One is real-time constraints: such systems and platforms operate on a schedule that can only tolerate limited delays before data is lost. The other is energy considerations. Anonymisation requires systems security to win its users' trust. A key fob is a case in point. The car identity is hidden inside a completely autonomous, sealed and battery-powered device, sometimes with an integral battery which cannot be replaced, and the car user is reassured that the fob is as good as, if not better than, a mechanical key. Similarly one could implement an ARUP anonymiser as a key-fob device with a non-replaceable battery. The user need not trust it, if the device has no radio emissions (which can be ensured by putting it in a sufficient Faraday cage). The host system (for example a car) could exchange data with it optically. Another trust boosting measure would be an arrangement whereby ARUP random data is supplied by the host system alongside input messages to Steps 1 and 5: this excludes the possibility of a compromised fob signalling its identity through a deliberate choice of supposedly random data: ϕ or ν, since those are revealed to the servers in the open at some point of the protocol. The rest of the data from the fob are transparent and externally checkable without accessing its secrets, so the fob's byzantine behaviour can be ruled out reliably.

We anticipate a device implementing ARUP on the client side to be similar to a *passive* key fob: the device receives a Step input message for Step 1, 3, 5 or 7, does the Step calculations and then emits an appropriate output message, returning to 'sleep' until the next message is received. Since all communication on ARUP is broadcast in the open, the ARUP fob can use a broadcast channel without security risk. The rest of the work: exchanging messages with RS/RA and compiling the sensor report, can be done by the host system, such as the car, which has no IoT constrains.

This approach gives us a natural way of evaluating the performance of the ARUP as an IoT-orientated protocol. We pose the following question: how many rounds of the protocol can be supported by a sealed key-fob implementation of it using a nonreplaceable battery to thwart side-channel attacks via measured power consumption? We could also turn the answer into the life span of the fob by assuming that a client will not perform more than 400 rounds a day (25 rounds per hour for 16 hours).

Example set-up. Consider a system that consists of an ESP32 with the WiFi/BT turned off, powered by a key-fob battery CR2477N and communicating with the host via an IrDA device HDSL-3211. To accommodate high-power bursts of the infrared link and the processor, the battery is bolstered by a parallel 0.22F supercapacitor FCS0V224ZFTB24. The parameters of all these devices are presented in Table 4. The choice of IrDA over BT or Zigbee is due to the fact that it is non-radio and so can easily be shielded or operated inside a Faraday cage; it has negligible power consumption on receive, and a sufficient communication speed (we chose circa 1Mbit/sec) on both receive and transmit. The battery leakage can be ignored, but the capacitor leakage should be accounted for as well as the power drain from ESP32 in deep sleep. The energy consumed for one round of the protocol is made up of the energy required by the ESP32 at the maximum current (but with WiFi and BT off) for the duration of the round computations and the energy dissipated by the IrDA device when it transmits the four output messages. Note that since the report R is received by the fob from the client's host, there is no need to retransmit it back at the end of the Step 5 output message; as this is the only variable field, we can calculate the output volume for one round reasonably accurately (assuming that the additional data required for CRC or FEC is only a few bytes). One can verify by examining the protocol pseudocode presented in Section 4.1 that the total output is less than 3300 bytes assuming the larger RSA modulus of 4K bits.

The figures for the power budget are presented at the bottom of Table 4. A few details stand out. First of all, the energy consumption of the computation is about 1/3 of that of the communication per round, standing at 311 to 955 nAh. This is thanks to the use of our original blind signature with limited public metadata. As follows from the data presented in Table 1, a full domain exponentiation alone would have been over 30 times the energy burn of the whole round, to say nothing of the multiplication and hashes. Computation would dominate the budget, as well as pushing the latency of the protocol exchange up to the whole-second scale. Further, looking at the bottom of the table, we can see that 750K rounds of the protocol are possible at 400 rounds per day without replacing the battery. The overall passive drain for one day is 264μAh compared to the circa 1.3μAh cost of a single round, which means

Table 4: Power Budget and Lifetime Analysis for ARUP executing on a battery-operated ESP32 with HSDL-3211 optical link

Parameter	Value	Unit	Energy (mAh)	Comments
ESP32 Computation (per round)				
Active current (radio off)(Espressif Corp, 2025)	40	mA		240 MHz, Wi-Fi/BT disabled
Computation time	28	ms		per round
Energy per round			0.000 311	mAh
HSDL-3211 Optical Link(Lite-on Corp, 2007) (per round)				
Data volume	3300	bytes		26,400 bits
Data rate	1.152	Mbps		HSDL-3211 maximum
Transmit time	22.92	ms		26,400 bits ÷ 1.152 Mbps
Transmit current	150	mA		Peak pulsed current
Energy per round			0.000 955	mAh
Total Per round				
Total energy			0.001 266	mAh
Quiescent Power (24-hour period)				
HSDL-3211 shutdown current	0.001	µA		1 nA typical
ESP32 deep sleep current	10	µA		Typical deep sleep
Supercapacitor leakage	1	µA		FCS0V224ZFTBR24 typical
Time period	24	hours		
HSDL-3211 shutdown energy			0.000 024	mAh
ESP32 deep sleep energy			0.240	mAh
Supercapacitor leakage energy			0.024	mAh
Total daily quiescent			0.264	mAh
Battery (CR2477N(Renata SA Corp, 2019))				
Capacity	950	mAh		Rated at 20°C
Max continuous current	2.5	mA		
Max pulse current	15	mA		For short pulses (<1s)
Supercapacitor (FCS0V224ZFTBR24)(KEMET Electronics Corp, 2026)				
Capacitance	0.22	F		220,000 µF
Rated voltage	3.5	V		
Voltage drop during Tx	0.0156	V		$\Delta V = (I_{TX} \times t_{TX})/C$
System Lifetime				
Rounds per day	400	day^{-1}		
Active energy per day			0.506	mAh
Quiescent energy per day			0.264	mAh
Total daily energy			0.770	mAh
Theoretical lifetime	1234	days		C_{BAT} ÷ daily total
	3.4	years		At 400 rounds/day
Rounds per battery	750 000	rounds		C_{BAT} ÷ E_{TOT}

`RA.py`	**Class `RA`.** An instance of this class is a Registration Authority. It is initialised with a single optional parameter `cache_lim`, default value 100, which defines the number of cache slots for storing input messages to prevent re-calculation. `onboard(z)` add a client with the chaintop `z`; `Step2(M1)` perform Step2 computation on input message M1; `Step4(M3)` perform Step4 computation on input message M3.
`RS.py`	**Class `RS`.** An instance of this class is a Reputation Server. It is initialised with a single optional parameter `cache_lim`, default value 100, which defines the number of cache slots for storing input messages to prevent re-calculation. `Step6(M5)` perform Step6 computation on input message M5; `Step8(M7)` perform Step8 computation on input message M7.
`client.py`	**Class `client`.** An instance of this class is a client. It is initialised with a single optional parameter `chlen`, default value 100, which defines the the length of the Winternitz chain pre-calculated in the process of creating the object. The following methods are defined: `report(rep)` set the situation report for the current round of the protocol to `rep`; `Step1(M8)` perform Step1 computation on input message M8; `Step3(M2)` perform Step3 computation on input message M2; `Step5(M4)` perform Step5 computation on input message M4; `Step7(M6)` perform Step7 computation on input message M6.

NB: all Step functions return the output message of the corresponding Step. All messages are instances of class `Message`.

Table 5: Reference implementation. Main components.

that circa 200 rounds per day makes the key fob 50% loaded. The exact figures are not important, but it is true that ARUP is in fact so efficient that its energy cost is comparable with the quiescent current drain of the platform. As the figures in Table 4 suggest, in our example the ARUP key fob can be used for 3 years without battery replacement. This substantiates our claim that it lends itself nicely to a secure IoT implementation. Coupled with the zero-trust property stemming from external supply of random numbers, this makes ARUP a *practical* security solution for an open anonymous reputation update environment.

6.2 Reference code

Needless to say, the above analysis is an *example*, albeit detailed enough, of a practical application of ARUP; it is essentially a feasibility study. If the protocol is to be used this way, there will be a need to test and validate its implementation in an embedded system. With this in mind, we have deployed reference python code in the public domain (comqas, 2026a) so as to facilitate validation of input and output messages for unit testing of the ARUP client and server. The structure of the code is presented in Tables 5 and 6.

`RUP.py`	Functions `Adj()` and `Upd()`. `Adj(a, sigma)` uses the current context of a user and the RS-proposed new reputation `a` to adjust the user's reputation and update the user's context. `Upd(a, R)` evaluates report R and returns a proposed new reputation Both functions are plug-ups. The implementor must supply their own version.
`hash_based.py`	Functions `H()` and `H_bar()`
`consts.py`	Definitions of `g`, `eps()`, `t`, and `t_hat`
`ARUP_message.py`	Class `Message` *constructor* create a message object from fields. Positional parameters are the fields and the key parameter is the format specifier `fmt=`: a string of characters composed of letters S, M and L; `extract(fmt)` return fields of the message object using the format string `fmt` to locate them.

Table 6: Reference implementation. Auxiliary components.

7 Related work

The problem of reputation-guided crowdsensing with anonymous reputation update is receiving continual attention as evidenced by undiminishing stream of publications focused on various aspects of it in the context of data gathering in automotive and smart-city scenarios. These works can be subdivided into two categories: research focussing on the security aspects of crowdsensing, including anonymity and privacy concerns, and research on prediction of the quality of the sensor reports and methods of reputation update. The latter category (see. e.g. paper (Zhang et al., 2025)) is outside the scope of this paper as we deal with the problem of anonymity/unlinkability and we approach it from the cryptographic perspective. The former category is no less pursued, with the main theme being the use of blockchain as an anonymisation mechanism. We cite papers (Xu et al., 2023), (Zheng et al., 2025) and (Li et al., 2025) as notable representatives of this line of research. Our own work is confined to IoT swarms, where crowdsensing units are low-power autonomous platforms, which must economise the computational cost while still achieving the security objectives. In particular, public key cryptography for the purpose of digital signature for a blockchain involves algorithms, the cost of which exceeds that of modular exponentiation, see the aforecited papers for details. Also, blockchain only provides pseudonymity and not unlinkability when taken per se.

Guy Fawkes protocols were first introduced by Anderson et al (Anderson et al., 1998) as a one-time digital signature, but there was not much work done to follow these ideas until article (Shafarenko, 2021), where they were laid into the foundations of a blockchain protocol suite. Our use of Guy-Fawkes message linking and piggybacking continues the same line of thought. The use of a Winternitz chain as a replacement of a fixed secret identity was discussed in a recent paper (Shafarenko, 2024). An alternative would be to introduce a Diffie-Hellman style protected channel to present to the RA a pseudonymous identity for linking coupons together. This would be at the expense of full-domain exponentiation on behalf of the client, whereas a Winternitz chain only requires a single hash calculation for client authentication. Also, the way we have introduced the Guy Fawkes technique into ARUP makes it possible to have a *fully public* RA, since the Authority does not store any secret at all with the exception of its own private key. That key is only used for signing, and the signing in our approach does not involve randomisation; this makes it possible for an external observer that watches network traffic to verify that the signature is not only correct (that is done by applying the appropriate public key) but that it contains no hidden information. Compared to all cited work, this gives a truly zero-trust solution to the unlinkability problem.

The RSA blind signature with public metadata has been studied in a few publications. The seminal work done in (Abe and Fujisaki, 1996) paved the way towards public metadata in blind signatures, and there already we find account of an attack that is enabled by an arbitrary choice of the set of public exponents for encoding metadata. This has been refined by the authors of (Amjad et al., 2025) to an algebraic constraint 2, but both papers agree that the public exponents that encode metadata should

be large to avoid constraint violations. The authors of (Abe and Fujisaki, 1996) use a hash function of the metadata directly and claim that the hash image should be at least half the bit length of the RSA modulus. In (Amjad et al., 2025) a salted hash is used and the salt value is precomputed to limit the probability of algebraic constraint violation, but in both approaches a long hash length is used, necessitating general modular exponentiation, which is expensive for low-power IoT devices. It is also important to note that the seminal work (Wang et al., 2013) does not use public metadata in a blind signature at all, instead passing the reputation value enciphered in a different, loosely linked copy of a coupon; this forces the client to trust the server not to include in the ciphertext any linkable data (e.g., location). By contrast our zero-trust scheme makes such attacks impossible.

Our construction is designed with smart sensors in mind, where the execution of modular multiplication for a 2Kbit modulus takes of the order 100μs and the duration of general modular exponentiation (in other words, RSA signing time) is commensurate with 1 sec, see Table 1, a difference of four orders of magnitude[2]. We therefore require a scheme whereby the client does not need general RSA exponentiation. As regards the cost of hash function calculation, it is typically in single microseconds and could be ignored compared to modular multiplication.

We believe the work done by Chien et al (Chien et al., 2001) to be very promising in this regard as the authors managed to build a scheme which utilises a small number of modular multiplications to provide a blind signature with public metadata not based on encoding the latter in the public exponent. Unfortunately there were some doubts about the security of their solution (Hwang et al., 2003), and although the doubts were eventually dispelled (Wen et al., 2005), it is quite clear to us that a more formal security proof for the method in (Chien et al., 2001) is still required.

We chose instead to specialise the approach in (Amjad et al., 2025) for an extremely small set of allowable metadata by putting each metadata value in one-to-one correspondence with a small prime number rather than a large pseudorandom number (as in (Amjad et al., 2025)). Our approach has no benefit if the metadata set is large enough to support qualitative data, e.g. verbal descriptions, since general exponentiation would become unavoidable either way, and since hash functions are fast to compute. However, for a set of four metadata values (levels 0–3 of reputation) we manage to replace modular exponentiation by a few tens of modular multiplications due to the smallness of all the exponents that the client has to use. Our method is also different in that it does not require the requester to agree the metadata with the signer first: our choice of a blinding factor allows a one step transaction: the requester provides the document to be signed blindly, and the signer works out the public metadata, signs the blinded document and announces the metadata. In our modification of (Amjad et al., 2025), the blinding factor is good for not one, but a few metadata values allowing the requester to unblind the signed document no matter which one of the values has been selected by the signer.

8 Conclusions

We have presented a new low-cost anonymous reputation update protocol ARUP intended for low-power, mobile IoT clients. Our approach reliably protects the client's anonymity and prevents sensing reports that include the location and time of observation from being linked up to reconstruct the client's route. As a result the identity of the user cannot be learned either directly (thanks to the pseudonymity of the client's temporary identity) or indirectly, by analysing the set of linked routes. The low-power property of the protocol stems from the fact that it requires only small-exponent modular exponentiation and a small number of hash computations. Even division, which is required for unblinding an RSA signature, is itself blinded and requested from a server, which reduces its cost to the client to that of two multiplications, whereas ordinarily it would be on a par with full size exponentiation.

The protocol has been evaluated on a real-world IoT platform both in terms of its timing, and on its energy consumption for an example implementation as a sealed key fob using off-the-shelf components. We have established that the fob can run on a single disc-type battery for three years while providing 400 rounds of the protocol per day (i.e. 400 sensing reports). We also established that the protocol latency on our platform to be within 30ms per round, which easily satisfies the requirements of a moving sensor in smart city and similar applications. We conclude that our proposed solution is both practical and economic, and we provide a reference implementation in Python to facilitate future product development.

[2]This is consistent with the average cost of modular exponentiation as circa $3|n|/2$ modular multiplications, where $|n|$ is the length of the RSA modulus in bits.

Acknowledgement

This research was supported by EU Project SMARTEDGE under Grant No 101092908.

References

Abe, M., Fujisaki, E., 1996. How to date blind signatures, in: Kim, K., Matsumoto, T. (Eds.), Advances in Cryptology — ASIACRYPT '96, Springer Berlin Heidelberg, Berlin, Heidelberg. pp. 244–251.

Amjad, G., Yeo, K., Yung, M., 2025. RSA blind signatures with public metadata. Proceedings on Privacy Enhancing Technologies 2025, 37–57. doi:10.56553/popets-2025-0004.

Anderson, R., Bergadano, F., Crispo, B., Lee, J.H., Manifavas, C., Needham, R., 1998. A new family of authentication protocols. SIGOPS Oper. Syst. Rev. 32, 9–20. URL: https://doi.org/10.1145/302350.302353, doi:10.1145/302350.302353.

Chien, H.Y., Jan, J.K., Tseng, Y.M., 2001. RSA-based partially blind signature with low computation, in: Proceedings. Eighth International Conference on Parallel and Distributed Systems. ICPADS 2001, pp. 385–389. URL: https://ieeexplore.ieee.org/document/934844, doi:10.1109/ICPADS.2001.934844.

comqas, 2026a. ARUP kit. URL: https://github.com/comqas/ARUP_kit.

comqas, 2026b. ESP32 accelerator benchmarking experiments: modular arithmetic and hash functions. URL: https://github.com/comqas/ESP32-accelerator-experiments.

Espressif Corp, 2025. ESP32 Series Datasheet. Espressif Systems. URL: https://documentation.espressif.com/esp32_datasheet_en.html.

Gennaro, R., Halevi, S., Rabin, T., 1999. Secure hash-and-sign signatures without the random oracle, in: Proceedings of the 17th International Conference on Theory and Application of Cryptographic Techniques, Springer-Verlag, Berlin, Heidelberg. p. 123–139.

Gidney, C., 2025. How to factor 2048 bit rsa integers with less than a million noisy qubits. arXiv preprint arXiv:2505.15917 URL: https://arxiv.org.

Gidney, C., Ekerå, M., 2021. How to factor 2048 bit rsa integers in 8 hours using 20 million noisy qubits. Quantum 5, 433. URL: https://doi.org.

Gu, B.j., Zhou, Y., Wang, W.n., 2009. Batch rsa signature scheme. Journal of Shanghai Jiaotong University (Science) 14, 290–292. URL: https://doi.org/10.1007/s12204-009-0290-1, doi:10.1007/s12204-009-0290-1.

Hwang, M.S., Lee, C.C., Lai, Y.C., 2003. Traceability on RSA-based partially signature with low computation. Applied Mathematics and Computation 145, 465–468. URL: https://www.sciencedirect.com/science/article/pii/S0096300302005003, doi:https://doi.org/10.1016/S0096-3003(02)00500-3.

Ji, W., Han, K., Liu, T., 2023. A survey of urban drive-by sensing: An optimization perspective. Sustainable Cities and Society 99, 104874. URL: https://www.sciencedirect.com/science/article/pii/S2210670723004857, doi:https://doi.org/10.1016/j.scs.2023.104874.

KEMET Electronics Corp, 2026. FC Series Supercapacitors Datasheet. KEMET Electronics Corporation. URL: https://content.kemet.com/datasheets/KEM_S6011_FC.pdf.

Li, Z., Yao, Y., Liu, A., Xiong, N.N., Zhang, S., Vasilakos, A.V., 2025. Reapp: A low-cost and accurate reputation evaluation based anonymous privacy preserving scheme in mobile crowdsourcing. Information Sciences 712, 122110. URL: https://www.sciencedirect.com/science/article/pii/S0020025525002427, doi:https://doi.org/10.1016/j.ins.2025.122110.

Lite-on Corp, 2007. HSDL-3211: IrDA Data Compliant Low Power 1.15 Mbit/s Infrared Transceiver. Lite-On. URL: https://media.digikey.com/pdf/Data%20Sheets/Lite-On%20PDFs/HSDL-3211.pdf.

Narkhede, N., Shapira, G., Palino, T., 2017. Kafka: The Definitive Guide Real-Time Data and Stream Processing at Scale. 1st ed., O'Reilly Media, Inc.

Renata SA Corp, 2019. CR2477N Lithium Battery. Renata SA. URL: https://www.renata.com/en-us/products/lithium-batteries/cr2477n/.

Sadiah, S., Nakanishi, T., 2022. An efficient anonymous reputation system for crowdsensing. Journal of Information Processing 30, 694–705.

Sadiah, S., Nakanishi, T., 2024. A distributed anonymous reputation system for v2x communication, in: 2024 IEEE International Conference on Consumer Electronics (ICCE), IEEE. pp. 1–6.

Shafarenko, A., 2021. A PLS blockchain for IoT applications: protocols and architecture. Cybersecurity 4, 4. URL: https://doi.org/10.1186/s42400-020-00068-0.

Shafarenko, A., 2024. Winternitz stack protocols for embedded systems and IoT. Cybersecurity 7, 34. URL: https://cybersecurity.springeropen.com/articles/10.1186/s42400-024-00225-9.

Wang, O., Xinlei, Cheng, W., Mohapatra, P., Abdelzaher, T., 2013. ARTSense: Anonymous reputation and trust in participatory sensing, in: 2013 Proceedings IEEE INFOCOM, pp. 2517–2525. URL: https://doi.org/10.1109/INFCOM.2013.6567058, doi:10.1109/INFCOM.2013.6567058.

Welser, J.J., Orzechowski, M., et al., 2023. Estimating the Energy Requirements to Operate a Cryptanalytically Relevant Quantum Computer. Technical Report WRA2427-1. RAND Corporation. URL: https://www.rand.org.

Wen, H.A., Lee, K.C., Hwang, S.Y., Hwang, T., 2005. On the traceability on RSA-based partially signature with low computation. Applied Mathematics and Computation 162, 421–425. URL: https://www.sciencedirect.com/science/article/pii/S0096300304001043, doi:https://doi.org/10.1016/j.amc.2003.12.110.

Xu, L., Zhang, Y., Song, S., Zhu, L., 2023. Blockchain-based privacy-preserving reputation management for crowdsensing, in: Proceedings of the 2023 4th International Conference on Computing, Networks and Internet of Things, Association for Computing Machinery, New York, NY, USA. p. 833–841. URL: https://doi.org/10.1145/3603781.3603929, doi:10.1145/3603781.3603929.

Zhang, M., Ye, Q., Yuan, Z., Deng, K., 2025. A secure and efficient user selection scheme in vehicular crowdsensing. Scientific Reports 15, 19111. URL: https://doi.org/10.1038/s41598-025-02530-w.

Zheng, L., Feng, T., Xie, Z., Li, X., Su, C., 2025. Barm: Blockchain-assisted anonymous authentication and reputation management for mobile crowdsensing in internet of vehicles. IEEE Internet of Things Journal 12, 22463–22477. doi:10.1109/JIOT.2025.3552447.